

Rapport : Echoes

Application mobile narrative développée avec React Native et Expo

Réalisé par : Clément Simon

Cours : Projet d'intégration & de développement

Professeur : Kamal Belouh

Ecole : IETCPS

Classe : BAC 3 informatique

Année Scolaire : 2025-2026

Date de remise : 28/05/2026

Plan du rapport

1. Rappel du sujet
2. Spécifications des choix et fonctionnalités implémentées
3. Justification technique
4. Architecture
5. Auto-évaluation
6. Problème rencontrés
7. Conclusion

1. Rappel du sujet

Echoes est une application mobile narrative développée avec React Native et Expo.

L'application prend la forme d'une conversation SMS interactive entre l'utilisateur et Julie (IA), une jeune femme piégée dans une crevasse. Le joueur communique avec elle via une interface de chat et ses messages influencent progressivement son état émotionnel, sa confiance, sa situation physique et le déroulement de l'histoire.

L'objectif du projet est de proposer une expérience immersive où la discussion ne se limite pas à un échange classique : Julie peut être disponible, occupée, endormie, revenir plus tard, ou laisser certains messages en attente avant d'y répondre.

Le projet met donc en place les bases d'un jeu narratif conversationnel avec persistance locale, gestion d'état, historique de messages et réponses générées par IA.

2. Spécifications des choix et fonctionnalités implémentées

Fonctionnalités de conversation

Fonctionnalité	Description
Écran d'accueil des conversations	Affiche la discussion principale avec le nom du contact et le dernier message reçu.
Écran de chat	Affiche l'historique complet des messages sous forme de conversation.
Envoi de message utilisateur	L'utilisateur peut écrire et envoyer un message à Julie.
Réponses de Julie par IA	Les réponses sont générées via un modèle IA appelé par OpenRouter.
Indicateur d'écriture	L'interface indique quand Julie est en train d'écrire.
Historique local	Tous les messages sont sauvegardés localement en SQLite.
Affichage de l'heure	Chaque message affiche l'heure d'envoi.
Indicateur de lecture	Les messages utilisateur affichent visuellement s'ils ont été traités par Julie.
Message d'introduction	Au premier lancement, Julie envoie automatiquement un premier message d'appel à l'aide.

Fonctionnalités narratives

Fonctionnalité	Description
Stress de Julie	Le niveau de stress évolue selon les réponses générées.
Confiance de Julie	La confiance envers l'utilisateur évolue selon ses messages.
Situation physique	Julie peut être piégée ou libérer sa jambe.
Actions avec durée	Certaines réponses indiquent que Julie effectue une action pendant plusieurs minutes.
Phase awake	Julie est disponible et peut répondre.
Phase busy	Julie est occupée par une action et ne peut pas répondre immédiatement.
Phase asleep	Julie dort et les messages sont conservés en attente.
Phase finalTwist	Une fin narrative spéciale peut être déclenchée.
Messages de retour	Julie peut envoyer un message lorsqu'elle revient d'une action ou du sommeil.
Messages en attente	Les messages envoyés pendant une indisponibilité sont traités plus tard.

Fonctionnalités de démonstration

Commande	Effet
##reset	Réinitialise la partie et supprime les messages.
##twist	Déclenche immédiatement le twist final.
##awake	Force Julie à redevenir disponible.
##busy	Force Julie à passer hors ligne pendant une courte durée.
##sleep	Force Julie à dormir.

Fonctionnalités de paramétrage

Fonctionnalité	Description
Modification du nom du contact	L'utilisateur peut modifier le nom affiché pour Julie.
Thème clair / sombre	L'utilisateur peut basculer entre deux thèmes visuels.
Sauvegarde locale des paramètres	Le nom du contact et le thème sont sauvegardés dans l'état du jeu.

Gestion des erreurs

Cas	Comportement
Réponse IA invalide	L'application tente de récupérer un JSON valide avant d'échouer.
JSON IA mal formé	Julie envoie un message réaliste indiquant un problème de réception.
Erreur API	Julie envoie un message indiquant que le signal est trop faible.
Message traité après erreur	Même en cas de fallback, le message utilisateur est marqué comme lu.

3. Justification technique

React native et expo

Le projet utilise React Native avec Expo afin de développer rapidement une application mobile.

Ce choix est adapté car Expo permet :

- un démarrage rapide du projet
- une compatibilité avec Android et iOS
- une utilisation simple avec Expo Go
- une bonne intégration avec TypeScript
- un accès facilité aux fonctionnalités natives nécessaires

TypeScript

TypeScript est utilisé pour sécuriser la structure des données principales.

Les types importants sont notamment :

- GameState
- Message
- ParsedAIResponse
- JuliePhase
- JulieSituation

Ce choix permet de mieux contrôler les flux de données, d'éviter certaines erreurs et de documenter le projet directement dans le code.

Expo Router

La navigation est gérée avec expo-router.

Les écrans principaux sont :

- index.tsx
- chat.tsx
- settings.tsx

Ce système rend la navigation simple et lisible, car la structure des fichiers correspond directement aux routes de l'application.

SQLite pour les messages

Les messages sont stockés avec expo-sqlite.

Ce choix est justifié car les messages forment une liste historique qui peut grandir avec le temps. SQLite permet de stocker, récupérer et trier ces messages efficacement.

Les messages sont sauvegardés avec :

- un identifiant
- un texte
- une date de création
- un indicateur utilisateur / Julie
- un indicateur de lecture par Julie

AsyncStorage pour le gameState

L'état global du jeu est stocké avec AsyncStorage.

Ce choix est adapté car le GameState est un objet unique qui contient :

- le thème
- le nom du contact
- la progression narrative
- le stress
- la confiance
- la phase actuelle de Julie
- les messages en attente

SQLite est utilisé pour les listes de messages, tandis qu'AsyncStorage est utilisé pour l'état global du jeu.

Context React pour l'état global

GameStateProvider expose l'état global du jeu à toute l'application.

Il fournit :

- gameState
- isGameStateLoading
- loadState
- saveGameState

Ce choix permet aux différents écrans et hooks d'accéder à l'état du jeu sans devoir passer les données manuellement de composant en composant.

Fetch

L'application utilise fetch pour appeler l'API OpenRouter.

Ce choix est volontaire :

- fetch est natif
- aucune dépendance supplémentaire n'est nécessaire
- l'appel API reste simple
- Axios aurait été utile pour une gestion plus complexe des requêtes, ce qui n'est pas nécessaire ici

Dans ce projet, fetch suffit largement pour effectuer une requête POST vers OpenRouter

OpenRouter et IA

Les réponses de Julie sont générées via OpenRouter.

Le prompt envoyé à l'IA contient :

- le rôle de Julie
- le contexte narratif
- l'état actuel du jeu
- l'historique récent des messages
- les contraintes de style d'écriture
- le format JSON obligatoire attendu en retour

L'IA doit répondre avec un JSON contenant notamment :

- le changement de stress
- le changement de confiance
- le message de Julie
- une durée d'action éventuelle
- une éventuelle évolution de situation

Parser IA séparé

Le parsing de la réponse IA est isolé dans aiResponseParser.ts.

Ce fichier :

- tente de parser directement la réponse
- nettoie les éventuels blocs markdown
- tente d'isoler le JSON entre accolades
- valide les champs obligatoires
- convertit le format IA vers le format interne du jeu

Cette séparation évite de mélanger l'appel réseau et la validation métier.

Séparation Hooks / Services / Repository/...

Le projet sépare clairement les responsabilités :

- les hooks orchestrent les flux React
- les services contiennent la logique métier ou technique
- les repositories gèrent la persistance
- les composants gèrent l'affichage

Cette séparation rend le projet plus lisible et réduit l'effet "spaghetti".

4. Architecture

Structure des dossiers

```
app/  
  _layout.tsx  
  index.tsx  
  chat.tsx  
  settings.tsx  
components/  
  ChatHeader.tsx  
  ChatInput.tsx  
  MessageBubble.tsx  
hooks/  
  GameStateProvider.tsx  
  useGameState.ts  
  useChat.ts  
  usePhaseManagement.ts  
  useIntroMessage.ts  
  useLastAssistantMessage.ts  
services/  
  aiService.ts  
  aiResponseParser.ts  
  databaseService.ts  
  demoCommandsService.ts  
  gameRulesService.ts  
  gameStateService.ts  
  messageRepository.ts  
  messageService.ts  
types/  
  GameState.ts  
  GameStateContextValue.ts  
  Message.ts  
  ParsedAIResponse.ts  
  DemoCommandAction.ts  
constants/  
  appConstants.ts  
  prompts.ts  
  theme.ts
```

Rôle des fichiers principaux

Fichier	Rôle
app/index.tsx	Écran d'accueil affichant la conversation et le dernier message.
app/chat.tsx	Écran principal de discussion avec Julie.
app/settings.tsx	Écran de configuration du contact et du thème.
components/ChatHeader.tsx	En-tête du chat.
components/ChatInput.tsx	Champ de saisie et bouton d'envoi.
components/MessageBubble.tsx	Affichage visuel d'un message.
hooks/useChat.ts	Orchestration du flux de conversation.
hooks/usePhaseManagement.ts	Gestion des phases de Julie.
hooks/useIntroMessage.ts	Envoi du premier message automatique.
hooks/useLastAssistantMessage.ts	Chargement du dernier message de Julie.
hooks/GameStateProvider.tsx	Fournit l'état global du jeu.
services/aiService.ts	Appel réseau vers OpenRouter.
services/aiResponseParser.ts	Validation et conversion de la réponse IA.
services/gameRulesService.ts	Règles pures du jeu.
services/messageService.ts	Création des objets message.
services/messageRepository.ts	Lecture et écriture SQLite.
services/gameStateService.ts	Lecture et écriture AsyncStorage.

Exemple flux 1 : envoi d'un message

Utilisateur écrit un message

- > ChatInput appelle handleSend
- > useChat reçoit le texte
- > le message utilisateur est créé
- > il est sauvegardé en SQLite
- > useChat vérifie si Julie est disponible
 - > si Julie est busy/asleep : le message est mis en attente
 - > si Julie est disponible : appel IA
- > aiService envoie le prompt et l'historique à OpenRouter
- > aiResponseParser nettoie et valide la réponse
- > gameRulesService calcule les modifications du GameState
- > useChat ajoute la réponse de Julie
- > le message utilisateur est marqué comme traité

Exemple flux 2: messages en attente

Utilisateur envoie un message pendant que Julie est indisponible

- > le message est sauvegardé
- > son id est ajouté à pendingMessageIds
- > quand Julie redevient awake
- > useChat traite les messages en attente
- > Julie répond
- > pendingMessageIds est vidé

Exemple flux 3 : les phases

usePhaseManagement vérifie régulièrement :

- > si le twist final doit être déclenché
- > si Julie doit revenir de busy
- > si Julie doit se réveiller
- > si un premier sommeil doit être planifié
- > si Julie doit commencer à dormir

La persistance

Messages

- > SQLite
- > messageRepository

GameState

- > AsyncStorage
- > gameStateService

Règles métier

Les règles de jeu sont centralisées dans gameRulesService.ts.

Exemples :

- isJulieAvailable
- shouldQueueForLater
- hasPendingMessages
- shouldWakeFromBusy
- shouldWakeFromSleep
- shouldStartSleep
- buildAssistantStateUpdates

Cela évite de répéter les mêmes conditions dans plusieurs fichiers.

5. Auto-Évaluation/ Fonctionnalités

Fonctionnalité annoncée	Réalisée	Commentaire
Application mobile React Native	Oui	Application développée avec Expo et React Native.
Navigation entre écrans	Oui	Accueil, chat et paramètres.
Interface de chat	Oui	Messages affichés sous forme de bulles.
Envoi de messages utilisateur	Oui	L'utilisateur peut écrire et envoyer un message.
Réponses générées par IA	Oui	Réponses générées via OpenRouter.
Prompt narratif contextualisé	Oui	Le prompt contient l'état du jeu et l'historique récent.
Sauvegarde des messages	Oui	Messages persistés avec SQLite.
Sauvegarde de l'état du jeu	Oui	GameState persisté avec AsyncStorage.
Premier message automatique	Oui	Julie envoie un premier message au lancement.
Gestion du stress	Oui	Le stress évolue selon la réponse IA.
Gestion de la confiance	Oui	La confiance évolue selon la réponse IA.
Situation physique de Julie	Oui	Julie peut passer de trapped à leg_freed.
Phases de disponibilité	Oui	Gestion de awake, busy, asleep, finalTwist.
Actions avec durée	Oui	Une réponse IA peut rendre Julie occupée.
Messages en attente	Oui	Les messages envoyés pendant l'indisponibilité sont traités plus tard.
Messages de retour	Oui	Julie peut envoyer un message après un retour de busy/sleep.
Commandes de démonstration	Oui	Commandes ##reset, ##twist, ##awake, ##busy, ##sleep.
Paramètres utilisateur	Oui	Modification du nom du contact et du thème.
Thème clair / sombre	Oui	Thème sauvegardé dans l'état du jeu.
Gestion des erreurs IA	Oui	Fallbacks réalistes et marquage des messages comme lus.
Parsing robuste de l'IA	Oui	Plusieurs tentatives d'extraction JSON sont prévues.
Architecture claire	Oui	Séparation entre hooks, services, repositories, types et composants.
Gameplay avancé	Partiel	Le gameplay reste basique et sert surtout de base technique.
Phases narratives prédéfinies	Partiel	Quelques phases existent, mais pas encore une vraie structure complète de progression.
Inventaire de Julie	Non	Non implémenté, mais identifié comme amélioration importante.
Objets utilisables	Non	Julie ne possède pas encore d'objets exploitables.
Choix à conséquences fortes	Partiel	Les choix influencent certains états, mais les conséquences restent limitées.
Évolution en arrière-plan sur tous les écrans	Partiel	Les phases sont surtout vérifiées pendant l'utilisation du chat.
Notifications push	Non	Non prévu dans le périmètre actuel.
Synchronisation cloud	Non	Données locales uniquement.

Gameplay encore basique

Le gameplay actuel reste volontairement très simple. Le projet met surtout en place la structure principale :

- conversation persistante
- réponses générées par IA
- états de Julie
- messages en attente
- sommeil
- indisponibilité
- sauvegarde locale

La progression narrative existe, mais elle reste limitée. L'histoire repose principalement sur quelques états simples :

- awake
- busy
- asleep
- finalTwist
- trapped
- leg_freed

Pour transformer ce prototype en vrai jeu narratif structuré, il faudrait enrichir fortement la partie gameplay.

Améliorations possibles

Les améliorations les plus importantes seraient :

- ajouter plusieurs phases narratives prédéfinies
- créer une progression plus contrôlée
- ajouter un inventaire pour Julie
- permettre à Julie d'utiliser certains objets
- permettre au joueur de guider Julie vers l'utilisation d'objets
- ajouter des obstacles concrets à résoudre
- créer des embranchements selon les choix du joueur
- ajouter des conséquences plus fortes aux décisions
- mieux suivre les actions déjà tentées
- éviter les répétitions narratives
- ajouter des conditions de réussite ou d'échec
- enrichir les interactions entre stress, confiance, inventaire et progression

Gestion des phases hors chat

Actuellement, la gestion des phases est liée au hook de chat. Cela signifie que les transitions sont principalement vérifiées lorsque le chat est ouvert.

Une amélioration future pourrait être d'ajouter un gestionnaire global de phases, monté au niveau de l'application, afin que le monde du jeu continue d'évoluer même lorsque l'utilisateur est sur l'écran d'accueil.

Cette amélioration demanderait cependant une refonte plus prudente afin d'éviter les timers doublons ou les messages de retour dupliqués.

6. Problèmes rencontrés

L'une des principales difficultés du projet a été de construire une architecture claire et maintenable. Le résultat n'est pas encore parfait, et certaines parties pourraient encore être améliorées, mais ce travail m'a surtout fait comprendre à quel point la réflexion autour de l'architecture est essentielle dans le développement d'une application.

Au départ, certaines responsabilités étaient trop mélangées entre les hooks, les services et les composants. Il a donc fallu prendre du recul pour mieux séparer l'interface, la logique métier, le stockage local et les appels à l'IA. Cette étape m'a montré que le code ne vient finalement qu'en second temps : avant d'écrire une fonction, il faut comprendre où elle doit se trouver, quel rôle elle doit avoir, avec quelles autres parties elle communique, et comment éviter qu'elle rende le projet difficile à maintenir.

Cette réflexion a mené à une organisation plus propre, avec des fichiers spécialisés comme `useChat.ts` pour la gestion de la conversation, `usePhaseManagement.ts` pour les phases de Julie, `messageRepository.ts` pour SQLite, `aiService.ts` pour l'appel à l'IA et `gameRulesService.ts` pour les règles du jeu. Même si cette architecture reste perfectible, elle rend le projet plus lisible et permet de mieux comprendre le rôle de chaque fichier.

Un autre problème important a été la gestion du flux asynchrone. Lorsqu'un utilisateur envoie un message, celui-ci doit être affiché immédiatement, sauvegardé en base locale, envoyé à l'IA, puis suivi d'une réponse qui peut aussi modifier l'état du jeu. Il fallait donc réfléchir à l'ordre des opérations pour éviter des incohérences entre l'affichage, SQLite et l'état global. Cette partie m'a montré que le plus difficile n'est pas toujours d'écrire le code (surtout maintenant avec l'IA...), mais de construire une logique cohérente et fiable.

La gestion des phases de Julie a aussi demandé une attention particulière. Julie peut être disponible, occupée, endormie ou dans la phase finale. Lorsqu'elle est busy ou asleep, les messages de l'utilisateur ne doivent pas être perdus : ils sont affichés directement, mais stockés dans une file d'attente via `pendingMessageIds`. Quand Julie redevient disponible, ces messages sont ensuite traités par l'IA. Cette logique a nécessité une bonne synchronisation entre la gestion des phases et la gestion du chat.

L'intégration de l'IA a également posé des contraintes, car le modèle doit répondre dans un format précis. Pour éviter qu'une réponse invalide bloque l'application, un système de parsing et de validation a été ajouté. Des messages de secours permettent aussi de garder une expérience cohérente en cas d'erreur API ou de réponse mal formée.

Enfin, il a fallu trouver un équilibre entre amélioration technique et stabilité. Certaines refontes plus importantes auraient pu rendre l'architecture encore plus propre, mais elles auraient aussi augmenté le risque de régression avant le rendu. Le choix a donc été de privilégier des améliorations ciblées, en appliquant au mieux les principes vus en cours : séparation des responsabilités, limitation des duplications, fonctions plus lisibles et code plus facile à maintenir.

Au final, ce projet m'a montré, une fois de plus, que développer une application ne consiste pas seulement à écrire du code. Le plus important est souvent de penser la structure, les responsabilités et les flux avant de coder. Une bonne architecture ne rend pas automatiquement le projet parfait, mais elle permet de le faire évoluer plus facilement, de mieux comprendre les erreurs et de garder une base plus saine pour la suite.

7. Conclusion

Le projet Echoes pose les bases d'un jeu conversationnel narratif.

La partie technique est fonctionnelle :

- interface mobile
- navigation
- stockage local
- gestion d'état
- historique de messages
- intégration IA
- parsing sécurisé
- phases de disponibilité
- messages en attente

Le projet est aujourd'hui structuré de manière globalement claire, avec une séparation entre les écrans, les composants, les hooks, les services, les repositories et les types. Cette organisation permet de mieux comprendre le rôle des différentes parties de l'application et facilite l'évolution du projet.

Cependant, le code pourrait encore être amélioré dans son ensemble. Certaines responsabilités pourraient être mieux harmonisées entre les fichiers, notamment entre les hooks, les services et la gestion de l'état global. Certains flux sont fonctionnels, mais restent parfois un peu complexes ou peu intuitifs, ce qui peut rendre la maintenance plus difficile à long terme. Une refonte partielle de l'architecture permettrait de mieux séparer la logique métier, la persistance des données, la gestion des phases narratives et l'affichage.

La principale limite concerne aussi le gameplay, qui reste encore assez basique. La structure actuelle permet déjà de simuler une conversation immersive avec Julie, mais une version plus aboutie devrait enrichir la progression narrative avec des phases prédéfinies, un inventaire, des objets utilisables, des obstacles et des conséquences plus marquées.

En résumé, le projet constitue un prototype de jeu narratif conversationnel, avec une base technique fonctionnelle et extensible. Néanmoins, plusieurs aspects pourraient encore être revus, notamment l'harmonisation du code, la répartition des responsabilités, l'optimisation des flux, la simplification de certaines logiques internes et l'approfondissement du gameplay.